

A faster branch-and-bound algorithm for the earliness-tardiness scheduling problem

Francis Sourd · Safia Kedad-Sidhoum

Published online: 17 November 2007
© Springer Science+Business Media, LLC 2007

Abstract This paper addresses the one-machine scheduling problem with earliness-tardiness penalties. We propose a new branch-and-bound algorithm that can solve instances with up to 50 jobs and that can solve problems with even more general non-convex cost functions. The algorithm is based on the combination of a Lagrangean relaxation of resource constraints and new dominance rules.

Keywords One-machine scheduling · Earliness-tardiness · Lagrangean relaxation · Branch-and-bound

1 Introduction

A useful but difficult criterion in scheduling theory is the minimization of both earliness and tardiness costs. It comes from the “just-in-time” philosophy in management and production theory: an item should be delivered exactly when it is required by the customer. Therefore, both early and tardy delivery of a task with respect to its due date is penalized. Recent surveys on these problems have been published by Hoogetveen (2005, Sect. 4), Lauff and Werner (2004), and Gordon et al. (2002).

In this paper, we consider the basic single-machine earliness-tardiness problem, which is denoted by $1|r_j|\sum \alpha_j E_j + \beta_j T_j$ in the standard three-field notation. A set

$\mathcal{J}$ of n jobs $J_1, \dots, J_n$ with positive processing times $p_1, \dots, p_n$ has to be scheduled with no preemption on a single machine, machine idle time is allowed. For any j , J_j is available at time r_j and has a due date d_j . If the completion time of J_j , denoted by C_j , occurs after d_j , the job is late and a penalty $\beta_j(C_j - d_j)$ must be paid. Conversely, if $C_j < d_j$, the job J_j is early and the penalty is then $\alpha_j(d_j - C_j)$. The goal is to find a schedule that minimizes the sum of all the penalties. Although we focus on this earliness-tardiness problem, the algorithm presented in this paper is still valid in the more general case where the cost for scheduling J_j is given by some function $f_j(C_j)$ which may be non-convex. In particular, our algorithm can solve some extensions of the models presented in the literature: time windows (Esteve et al. 2006), due windows (Wan and Yen 2002), non-execution penalties (Hassin and Shani 2005), prohibition of idle time (Liaw 1999; Valente and Alves 2005). In the earliness-tardiness case, we simply have $f_j(C_j) = \max(\alpha_j(d_j - C_j), \beta_j(C_j - d_j))$, which is a convex piecewise linear function with two segments.

Since $1||\sum w_j T_j$ is strongly NP-hard (Lenstra et al. 1977), this problem is also strongly NP-hard (see also Garey et al. 1988) and, as observed in the many surveys (Hoogetveen 2005; Lauff and Werner 2004; Gordon et al. 2002; Baker and Scudder 1990), polynomial cases are very rare. Most of them assume that there is a common due date, that is $d_j = d$ for all jobs. The most general polynomial-time algorithm for a problem with distinct due dates is due to Verma and Dessouky (1998) and it can solve the case where $p_j = p$, $\alpha_1 \leq \dots \leq \alpha_n$, and $\beta_1 \leq \dots \leq \beta_n$. Hassin and Shani (2005) present a list of existing polynomial time algorithms for earliness-tardiness problems as well as some extensions.

F. Sourd (✉)
LIP6, CNRS, 4 place Jussieu, 75252 Paris Cedex 05, France
e-mail: Francis.Sourd@lip6.fr

S. Kedad-Sidhoum
LIP6, Université Pierre et Marie Curie, 4 place Jussieu,
75252 Paris Cedex 05, France
e-mail: Safia.Kedad-Sidhoum@lip6.fr

We are interested in an exact approach for the general case. The first branch-and-bound procedure was proposed by Fry et al. (1987) for the case $\alpha_j = \alpha$ and $\beta_j = \beta$ and was able to solve instances with up to 12 jobs. The special case $\alpha_j = \beta_j = 1$ is studied by Kim and Yano (1994) and Fry et al. (1996) who proposed algorithms viable for 20 and 25 jobs, respectively. Hoogeveen and van de Velde (1996) proposed six lower bounds and several dominance rules for the case $\alpha_j = \alpha$ and $\beta_j = \beta$ but could not solve instances with more than 25 jobs. For the same problem, Chen and Lin (2004) presented a long list of dominance rules and experiments for instances with 20 jobs. Finally, another algorithm was proposed by Chang (1999) for the symmetric case ($\alpha_j = \beta_j$) that could solve all the instances up to 35 jobs and some instances with 40 and 45 jobs—however, the parameters used in the generation of the instances indicate that these instances are rather easy.

For the general case, Sourd and Kedad-Sidhoum (2003) proposed a branch-and-bound procedure based on a pseudo-polynomial time lower bound based on an assignment problem. A similar lower bound was reported by Bülbül et al. (2007). This lower bound can be computed in $O(n^2T)$ time where T is the horizon of the schedule. This is significantly more time consuming than the previously proposed lower bounds but the lower bound is also significantly better. The resulting branch-and-bound algorithm is able to solve the instances with up to 30 jobs. Sourd (2004) showed that a polynomial-time lower bound can be derived from this approach but, unfortunately, it does not help to solve instances with more jobs. Tanaka et al. (2003) also failed to solve instances with even 30 jobs. Finally, we can mention the approach of Liaw (1999)—recently generalized by Valente and Alves (2005)—to solve the problem with no idle time inserted but, even in this easier case, instances with more than 30 jobs seem to be out of reach.

The contribution of this article is an improved algorithm that is capable of solving larger instances than before. Typically, in our experimental tests, almost all the 40-job instances and 85% of the 50 jobs instances are solved within 1000 seconds. This result is due to a stronger lower bound which takes even more time to compute than those developed by (Sourd and Kedad-Sidhoum 2003) and (Sourd 2004). It relies on a Lagrangean relaxation of the *time-indexed formulation* (see, e.g., De Sousa and Wolsey 1992). Two relaxed problems can be derived from this formulation. First, the number of occurrences of the jobs in the schedule can be relaxed. This approach was followed by Péridy et al. (2003) to minimize the number of late jobs and by Avella et al. (2005) to minimize the weighted completion time. The second approach, introduced by Fisher (1976) and Christofides et al. (1987), consists of relaxing the resource constraints. Much work was devoted to the resolution of the Lagrangean problem, the convergence of the subgradient and its usage to find good or optimal solutions (Luh

et al. 1990; Chen et al. 1998; Kaskavelis and Caramanis 1998; Möhring et al. 2003). However, to the best of our knowledge, all these results are related to multiple machine environments so that it was a surprise for us that this type of lower bound is also efficient in the simple single machine case without precedence constraints.

Lagrangean relaxations are usually time-consuming so that it is often challenging to use them in a branch-and-bound algorithm. The main achievement of our work is to efficiently integrate this lower bound in the search algorithm and to propose new dominance rules. Section 2 describes all the components of the branch-and-bound algorithms (initial upper bound, lower bound, dominance rules, and branching scheme). Experimental results and comparison to other approaches are presented in Sect. 3.

2 The branch-and-bound algorithm

2.1 Initial solution

The preliminary step of the algorithm is a *heuristic search* for a good solution in order to have a good upper bound for the optimum. Based on our previous experience (Sourd 2006), we came to the conclusion that a simple iterated descent algorithm with a generalized pairwise interchange neighborhood serves our purpose of quickly finding a good starting solution. By generalized pairwise interchange, we mean that the solutions are encoded by the job sequence and these sequences are modified by either a pairwise job swap, or an “extract and relocate” operation. The cost of the schedule when the sequence is modified can be efficiently computed using the ideas of Hendel and Sourd (2006).

As this simple heuristic was never compared to optimal solutions, Table 1 shows the performance of three variants of the algorithm tested on a set of 4914 instances whose optimum are known (see Sect. 3.1 for details about instance generation). HEUR1 is a simple descent algorithm from a *random* sequence. HEUR10 consists of iterating HEUR1 ten times, starting each time from a different random sequence, while HEUR n iterates HEUR1 n times, where n is the number of jobs of the instance. For each algorithm, the three lines respectively indicate the number of times the heuristic found the optimum, the mean deviation and the max deviation for each instance class.

The experiments confirm the high efficiency of this very basic heuristic search algorithm. Indeed, the performances of HEUR1 are not bad on average. However, while HEUR1 may return a bad local optimum (as indicated by the maximal deviation), HEUR10 and *a fortiori* HEUR n very often find the optimum and the mean deviation is small.

2.2 Lower bound

We first present the time indexed formulation of our problem. As mentioned in the introduction, the lower bound is

Table 1 Efficiency evaluation of the descent algorithms

		$n = 20$	$n = 30$	$n = 40$	$n = 50$	All
HEUR1	Optimum found	62.1%	38.4%	23.5%	17.4%	36.0%
	Mean deviation	1.06%	1.42%	1.46%	1.59%	1.38%
	Max deviation	21.9%	20.5%	22.3%	16.2%	22.3%
HEUR10	Optimum found	89.4%	75.8%	62.6%	55.7%	71.4%
	Mean deviation	0.12%	0.20%	0.26%	0.28%	0.21%
	Max deviation	8.76%	10.4%	6.47%	7.63%	10.4%
HEUR n	Optimum found	91.9%	86.3%	80.6%	76.4%	84.1%
	Mean deviation	0.08%	0.09%	0.10%	0.11%	0.10%
	Max deviation	8.76%	10.0%	5.53%	4.10%	10.0%

derived from a Lagrangean relaxation of this mixed integer program (MIP). We first introduce the horizon T which must be an upper bound on the makespan of an optimal schedule of the problem. In earliness-tardiness scheduling, we can choose $T = \max_j d_j + \sum_j p_j$. The notation $[t, t']$ is used to designate the set of the integers between t and t' . For any $t \in [r_j, T - p_j]$, let x_{jt} be a binary variable equal to 1 if the task J_j starts at time t and 0, otherwise. Let us define $est_j(t) = \max(r_j, t - p_j + 1)$ for $r_j \leq t \leq T - p_j$. $est_j(t)$ is the earliest start time of J_j such that it may be in process in time slot t .

$$\min \sum_{j \in \mathcal{J}} \sum_{t=r_j}^{T-p_j} f_j(t - p_j) x_{jt}, \quad (1)$$

$$\text{s.t.} \quad \sum_{t=r_j}^{T-p_j} x_{jt} = 1 \quad \forall j \in \mathcal{J}; \quad (2)$$

$$\sum_{j \in \mathcal{J}} \sum_{s=est_j(t)}^t x_{js} \leq 1 \quad \forall t \in [0, T - 1]; \quad (3)$$

$$x_{jt} \in \{0, 1\} \quad \forall j \in \mathcal{J}, \forall t \in [r_j, T - p_j]. \quad (4)$$

Equations (2) ensure that each job is executed exactly once. Inequalities (3), also referred to as *resource constraints*, state that at most one job can be handled at any time. Clearly, this formulation allows the occurrence of idle time. The objective (1) renders the cost of the schedule.

The lower bound is derived from the Lagrangean relaxation of the resource constraints (3) (Fisher 1976; Christofides et al. 1987). Our choice was motivated by the study of Kedad-Sidhoum et al. (2007) in the parallel machine case which shows that this Lagrangean relaxation gives better results than the Lagrangean relaxation of constraints (2). More precisely, even if both relaxations theoretically converge to the same lower bound, its convergence is

faster. Lagrangean multipliers $\mu = (\mu_0, \dots, \mu_{T-1}) \geq 0$ are introduced for each t so that, for each vector $\mu \geq 0$, a lower bound denoted by $\mathcal{L}(\mu)$ is obtained by solving the so-called Lagrangean problem:

$$\min_{x_{jt}} \sum_{j \in \mathcal{J}} \left(\sum_{t=r_j}^{T-p_j} f_j(t - p_j) x_{jt} + \mu_t \left(\sum_{s=est_j(t)}^t x_{js} - 1 \right) \right) \quad (5)$$

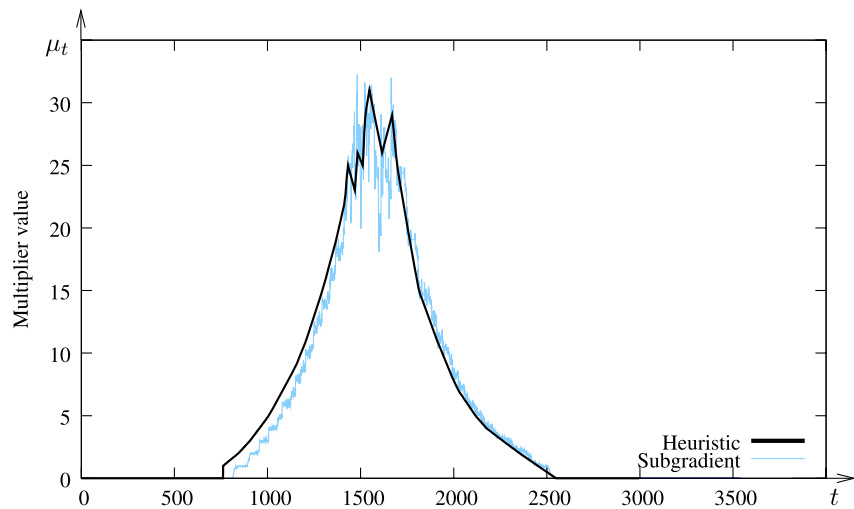
subject to the constraints (2) and (4).

Once the resource constraints are relaxed, the resulting Lagrangean problem consists of scheduling “unconstrained” tasks whose starting costs are defined by $c_{jt} = f_j(t - p_j) + \sum_{s=t}^{t+p_j-1} \mu_s$. Clearly, J_j must start at time t where c_{jt} is minimum. This start time can be computed in $O(T)$ time by observing that $c_{j,t+1} = c_{jt} + f_j(t + 1 - p_j) - f_j(t - p_j) - \mu_t + \mu_{t+p_j}$. Consequently, $\mathcal{L}(\mu)$ is computed in $O(nT)$.

Good multipliers have then to be computed in order to maximize $\mathcal{L}(\mu)$. This phase is usually performed through a subgradient method. Obviously, the convergence speed of this phase is of key importance because the computation of the lower bound is processed at each node of the branch-and-bound procedure. We now present a heuristic to find a priori good multipliers. This approach was used by Potts and coauthors for several scheduling problems with the objective of minimizing total or weighted tardiness (Potts and Van Wassenhove 1983, 1985; Potts 1985). In particular, Belouadah and Potts (1994) considered the Lagrangean relaxation of the resource constraints of a time-indexed formulation for the problem $P || \sum w_i C_i$. Comparatively, our heuristic takes into account the due dates and the earliness costs but it is limited to the single-machine case. However it could easily be extended to the parallel-machine case by using the *change time slots* of Belouadah and Potts.

The heuristic is based on the initial solution, denoted by σ , computed by the heuristic of Sect. 2.1. Roughly speaking, the multipliers are approximated by a piecewise linear function whose slopes depend on the job in process in σ .

Fig. 1 Heuristic and optimized Lagrangean multipliers



Algorithm 1: Heuristic to produce good Lagrangean multipliers

Input: a good feasible schedule σ
Output: good Lagrangean multipliers $\mu_0, \dots, \mu_{T-1}$
 $\mu_T = 0$
for $t = T - 1$ **downto** 0 **do**
 if a job is in process at t in σ **then**
 let J_j be this job in process
 if $t \geq d_j$ **then** $\mu_t = \mu_{t+1} + \beta_j/p_j$ **else** $\mu_t = \mu_{t+1} - \alpha_j/p_j$
 end
 $\mu_t = 0$
end

More precisely, they are initialized by Algorithm 1. This algorithm runs in $O(T)$ time, however, the underlying piecewise linear approximation can obviously be computed in $O(n)$ time and, using this piecewise linear representation of μ , we can derive a polynomial-time lower bound using the approach of Belouadah and Potts (1994). However, according to our preliminary tests, it is worth using the subgradient optimization with the T multipliers $\mu_1, \dots, \mu_T$ to improve this lower bound.

In order to give a theoretical justification of this heuristic algorithm, we provide the following proposition that is similar to the approach of Belouadah and Potts (1994).

Proposition 1 *In the case where $d_j = 0$ and $r_j = 0$ for all jobs (that is we have an instance of $1||\sum w_j C_j$), the heuristic gives the optimal multipliers when it is run with Smith's schedule (Smith 1956) as input schedule σ .*

Proof As $d_j = 0$, earliness penalties are, of course, irrelevant here. The jobs in Smith's schedule σ are sequenced in the non-decreasing order of the β_j/p_j ratios. For sake of simplicity, we assume that $\beta_1/p_1 > \beta_2/p_2 > \dots > \beta_n/p_n >$

0 but the result is still valid when two jobs have the same ratio β_j/p_j . In the optimal schedule σ provided by Smith's rule, the completion time of J_j is $C_j(\sigma) = \sum_{i=1}^j p_i$. Let μ be the vector of Lagrangean multipliers initialized by the heuristic. We show that each job j completes at time $C_j(\sigma)$ in the optimal Lagrangean solution. To this end, let us consider a solution for $\mathcal{L}(\mu)$ and let us assume that $x_{jt} = 1$ for some $t < C_j(\sigma) - p_j$. By modifying the solution such that $x_{j,t+1} = 1$ (and thus $x_{jt} = 0$), the cost increases by $c_{j,t+1} - c_{jt} = \beta_j + \mu_{t+p_j} - \mu_t = \beta_j + \sum_{\theta=t}^{t+p_j-1} (\mu_{\theta+1} - \mu_\theta)$. Since $\theta + 1 \leq t + p_j < C_j$, the job J_i in process at θ is such that $\beta_i/p_i \geq \beta_j/p_j$ so that $\mu_{\theta+1} - \mu_\theta = -\beta_i/p_i \leq -\beta_j/p_j$. Therefore, $\sum_{\theta=t}^{t+p_j-1} (\mu_{\theta+1} - \mu_\theta) \leq -\beta_j$, which proves that the cost increase is negative. Similarly, if $x_{jt} = 1$ for some $t > C_j - p_j$, changing the solution by $x_{j,t-1} = 1$ also results in a cost decrease. Therefore, an optimal solution is to set $x_{jt} = 1$ if and only if $t = C_j(\sigma)$. As this solution satisfies the relaxed resource constraints, it is both primal and dual optimal. $\square$

In practice, the heuristic gives good multipliers as illustrated by Fig. 1. The graph compares, for a typical instance, the multipliers found by the heuristic (the piecewise linear function) with the multiplier values obtained after the convergence of the subgradient method (the highly irregular curve). The multiplier curve of course varies with each instance, and especially with the parameters τ and ρ defined in Sect. 3.1. However, in all our observations, the heuristic piecewise linear curve and the "irregular" optimal curve are always correlated.

Such a multiplier initialization gives a very good lower bound after only a small number of iterations of the subgradient method (typically, the convergence loop is stopped after n iterations).

2.3 Branching scheme

The *branching scheme* builds the sequence of tasks from the first to the last one. In other words, at a node of depth k in the search tree, we have an initial subsequence σ of k tasks while the other $n - k$ tasks must be scheduled after all the sequenced tasks. The cost of the initial subsequence is modeled by a function Σ_σ which is called the *initial cost function*. More precisely, $\Sigma_\sigma(t)$ is the minimal cost such that all the tasks in the subsequence σ are completed before t . The definition domain of Σ_σ is $[C_{\max}(\sigma), T]$ where $C_{\max}(\sigma)$ is the earliest completion time (*makespan*) of the initial subsequence and T is the horizon. For ease of notation, we assume that $\Sigma_\sigma(t) = \infty$ for any t that is out of the definition domain.

When the cost function f_j is general, the values of Σ_σ must be computed for any integer time point in $[C_{\max}(\sigma), T - 1]$. In the earliness-tardiness case, Σ_σ is a convex, non-increasing, piecewise linear function with at most k segments (Sourd 2005), which can be more efficiently encoded in $O(k)$ space. More generally, Σ_σ is piecewise linear when the cost functions f_j are piecewise linear and Σ_σ is convex when the cost functions f_j are convex.

When a child node is created, a job—say J_j , which is not in σ —is added to the end of the sequence σ . This operation creates a new subsequence denoted by $\sigma \oplus J_j$. The new initial cost function $\Sigma_{\sigma \oplus J_j}$ is derived from Σ_σ by a very simple dynamic programming procedure. This operation takes $O(T)$ time in the general case, but only $O(k)$ time in the earliness-tardiness case (Sourd 2005).

At any node of the search tree, the computation of the lower bound must be adapted in order to take into account the presence of the initial subsequence σ . For a given set of multipliers λ , the lower bound is given by $\min_\theta (\Sigma_\sigma(\theta) + \mathcal{L}_\theta(\lambda))$ where $\mathcal{L}_\theta(\lambda)$ is the cost of the Lagrangean relaxation for scheduling the non-sequenced tasks in the interval $[\theta, T]$, that is we add the constraint $x_{jt} = 0$ for all non-sequenced jobs J_j and all $r_j \leq t < \theta$ (and we remove the variables related to sequenced jobs).

In order to compute $\mathcal{L}_t(\lambda)$, we compute, for all θ and all non-sequenced jobs J_j , the values $c'_{j\theta} = \min_{\theta \geq t} c_{j\theta}$ which is done in $O((n - k)T)$ time by using the relationship $c_{j,t+1} = c_{jt} - \mu_t + \mu_{t+p_j}$. Then, all the values $\mathcal{L}_t(\lambda)$ for $0 \leq t < T$ can be computed in $O((n - k)T)$ and, therefore, the lower bound is not longer to compute than the root lower bound.

Moreover, very good multipliers for a node can be heuristically derived by taking the multipliers of the parent node.

2.4 Dominance rules

2.4.1 Dominating initial cost function

Our *dominance rule* consists of checking whether the initial subsequence σ is dominated by another subsequence or

by the conjunction of several subsequences. Moreover, we also propagate some information about partially dominated schedules in order to restrict the set of possible completion times of some jobs and, by this way, to reinforce the lower bound.

Let $J(\sigma)$ be the set of jobs contained in σ . If the three following conditions are satisfied

- σ' is a subsequence such that $J(\sigma') = J(\sigma)$,
- $\Sigma_{\sigma'}(t) \leq \Sigma_\sigma(t)$ for all $t \geq 0$, and
- $\Sigma_{\sigma'}(t) < \Sigma_\sigma(t)$ for some $t \geq 0$,

then σ is dominated by σ' . Indeed, for any solution starting by the subsequence σ , we can get a better or equal solution by reordering the initial subsequence according to σ' . Therefore, the node associated to the subsequence σ can be discarded.

More generally, let S be a set of subsequences such that $\sigma \notin S$ and, for any $\sigma' \in S$, $J(\sigma') = J(\sigma)$. Let $\Sigma_S(t) = \min_{\sigma' \in S} (\Sigma_{\sigma'}(t))$. The dominance is formally proved in the next proposition.

Proposition 2 *Let S be a set of subsequences such that $\sigma \notin S$. If*

- $J(\sigma') = J(\sigma)$ for any $\sigma' \in S$,
- $\Sigma_S(t) \leq \Sigma_\sigma(t)$ for all $t \geq 0$, and
- $\Sigma_S(t) < \Sigma_\sigma(t)$ for some $t \geq 0$,

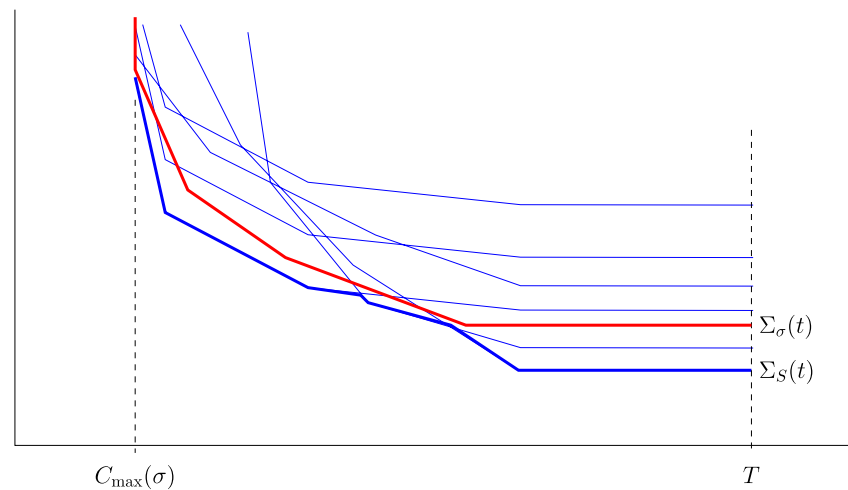
then σ is dominated by S , which means that for any solution starting by the subsequence σ , a better—or equal—solution can be derived by reordering the initial tasks according to some subsequence in S .

Proof Let us consider a feasible solution starting by the subsequence σ . Without loss of generality, we can assume that the jobs are sequenced in the order $J_1, J_2, \dots, J_n$. Let k be the number of jobs in the subsequence σ . Clearly, $\sigma(k) = J_k$. By construction, there exists a subsequence σ' in S such that $\Sigma_{\sigma'}(C_k) \leq \Sigma_\sigma(C_k)$. Therefore, we can schedule the jobs $J_1, \dots, J_k$ before C_k and let the completion times of the other jobs unchanged. Clearly, the cost of this new schedule is not greater than the initial one. $\square$

In our implementation, at any node, a set S of subsequences is heuristically built. It corresponds to a neighborhood of σ in which the last job of the subsequence is either re-inserted in an earlier position, or swapped with an earlier job. For instance, with $\sigma = (J_1, J_2, \dots, J_k)$ then we use

$$\begin{aligned} S(\sigma) = & \{(J_k, J_1, J_2, \dots, J_{k-1}), (J_1, J_k, J_2, \dots, J_{k-1}), \dots, \\ & (J_1, J_2, \dots, J_k, J_{k-1})\} \\ & \cup \{(J_k, J_2, \dots, J_{k-1}, J_1), \\ & (J_1, J_k, \dots, J_{k-1}, J_2), \dots, \\ & (J_1, \dots, J_k, J_{k-1}, J_{k-2})\}. \end{aligned}$$

Fig. 2 A dominated subsequence



As illustrated by Fig. 2, the so called *dominating initial cost function* $\Sigma_S(t)$ is a non-convex piecewise linear function. To prevent bad behaviors due to numerical instability, the dominance rule is activated only if $\Sigma_S(t) + \epsilon < \Sigma_\sigma(t)$ for some small positive ϵ .

In order to improve the dominating initial cost function and to avoid redundant calculations, the pair $(J(\sigma), \min(\Sigma_S, \Sigma_\sigma))$ can be stored—in a hash table in our implementation—so that the information about dominating schedules can be used at subsequent nodes. Note that this information is stored even if the current initial sequence is not dominated. Thus, when the branch-and-bound algorithm opens a node with an initial sequence σ^* , it first searches in the hash table for a pair of the form $(J(\sigma^*), \Sigma)$. If such a function Σ exists, we first check whether $\Sigma < \Sigma_\sigma$. If successful, this quick test proves that σ is dominated. Otherwise, we can take $\min(\Sigma, \Sigma_{S(\sigma^*)})$ as the dominating initial cost function, which is clearly better than $\Sigma_{S(\sigma^*)}$.

It must be observed that the storage of all these functions requires exponential space. For medium-size instances, computation times are significantly improved (by about 20%) but, for more difficult instances (typically instances with 50 jobs or more), memory limitation prevents this approach to work well, unless an elaborated policy for storage management is implemented.

2.4.2 Improving the initial cost function

We now show how the lower bound, and more precisely the initial cost function Σ_σ , can be improved when the current subsequence σ is not dominated by the dominating initial cost function Σ_S . The key idea is that, even if we do not have $\Sigma_S(t) < \Sigma_\sigma(t)$ for all t , there may be time intervals over which the inequality is satisfied. In such a case, the subsequence is said to be *partially dominated*. Without loss of generality, we assume again that $\sigma = (J_1, \dots, J_k)$.

Let us first assume that for some $\theta > 0$, we have that $\Sigma_S(t) < \Sigma_\sigma(t)$ for all $t \in [0, \theta]$. Thus, any schedule starting with σ such that $C_k < \theta$ is dominated. In the current node subproblem, we are not interested in finding solutions with $C_k < \theta$. Hence, we can restrict the definition domain of Σ_σ to be equal to $[\theta, T)$ or even to $[[\theta], T)$ if the r_i , d_i and p_i are integer. Clearly, this change in the current subproblem does not discard any dominating optimal solution and it may improve the quality of the lower bound since Σ_σ is used in the calculation of the lower bound.

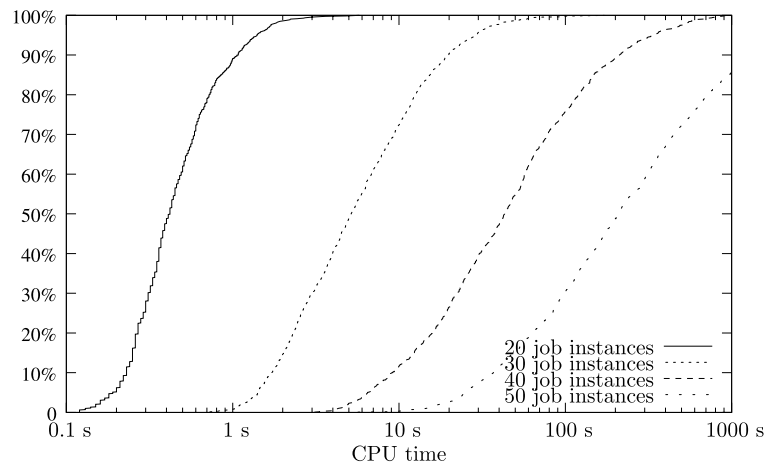
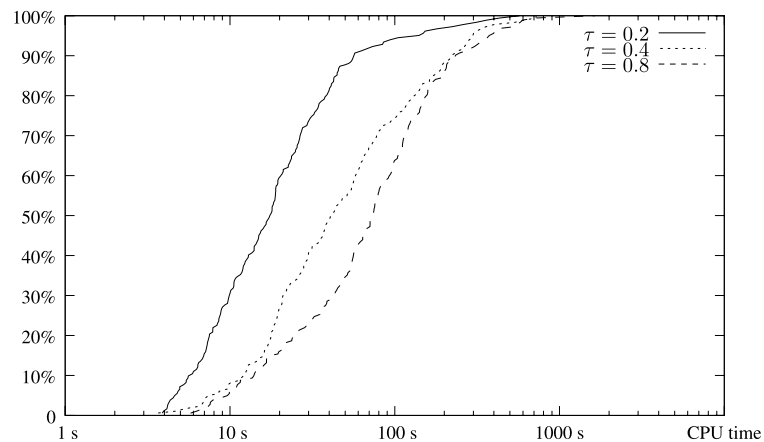
Similarly, we consider the case where, for some $\theta > 0$, we have that $\Sigma_S(t) < \Sigma_\sigma(t)$ for all $t \geq \theta$. In the current node, we are then interested only in the schedules such that $C_k \leq \theta$. We can then change the right end of the definition domain of Σ_σ to θ (or $[\theta]$).

More generally, we can remove from the definition domain of Σ_σ any interval on which Σ_σ is dominated by Σ_S . The computation of the lower bound in presence of Σ_σ , which is described in Sect. 2.3, can be trivially adapted in order to deal with these holes in the definition domain.

3 Experimental results

The algorithm was implemented in C++ (it can be compiled with MSVC7 or gcc 3.3). It is available on the Internet.¹ In the tested version, the initial heuristic is HEUR10 and the dominating initial cost functions are not stored for reasons given in Sect. 2.4.1.

¹<http://www-poleia.lip6.fr/~sourd/project/et>.

Fig. 3 Percentage of solved instances**Fig. 4** Runtime distribution of 40-job instances when τ varies

3.1 Instances

The generation scheme of the test instances is based on those used in the literature (Hoogeveen and van de Velde 1996; Sourd and Kedad-Sidhoum 2003). In order to avoid too many parameters, all the release dates are zero. The difficulty of instances with release dates is tackled at the end of this section (in Sect. 3.3). For a given value of n , the processing times of each job are first randomly drawn from the uniform distribution $U[10, 100]$. The due date are drawn from $U[d_{\min}, d_{\min} + \rho P]$ where $d_{\min} = \max(0, P(\tau - \rho/2))$ and $P = \sum_{j=1}^n p_j$. The two parameters τ and ρ are the *tardiness* and *range* parameters respectively.

We generated instances for $n \in \{20, 30, 40, 50\}$, $\tau \in \{0.2, 0.3, 0.4, 0.5, 0.6, 0.7, 0.8\}$, and $\rho \in \{0.2, 0.3, 0.4, 0.5, 0.6, 0.7, 0.8\}$. For each value of (n, τ, ρ) , 26 instances were generated, which induces a total of 5096 instances.

3.2 Results

As for any branch-and-bound algorithm, the computation times required to solve two different instances, even if generated with the same parameters, may vary a lot. Therefore,

the average computation time is not very relevant and we prefer to present our results with a runtime distribution diagram: for a given CPU time t on the log-scale abscissa axis, we read in ordinate the percentage of instances that were solved within the time t .

Figure 3 shows the runtime distributions to respectively solve the instances with 20, 30, 40, and 50 jobs. Clearly, this new algorithm outperforms previous exact algorithms whose limits are about 30-job instances: indeed, all the 40-job instances and more than 85% of the 50-job instances are solved within 1000s. The algorithms of Sourd and Kedad-Sidhoum (2003) and Sourd (2004) are able to efficiently solve instances with 30 jobs but the number of nodes is significantly more important. This observation explains why these algorithms fail to solve most instances with 40 jobs while our new algorithm succeeds.

Figure 4 compares the runtimes to solve the instances with 40 jobs for different values of the tardiness factor τ . It indicates that the instances are generally harder when the tardiness factor is bigger, which means that the due dates are distant. It can be explained by observing that the jobs scheduled after the last due date are ordered according to the ratio of p_i/β_i . Therefore, when the last due date is earlier, the

instance should be easier. Furthermore, we also observe that the optimal solution of the instances with $\tau = 0.8$ have some idle time inserted before the first task so we can conclude that instances with $\tau > 0.8$ are not more difficult. It roughly corresponds to the case of a *large* or *unrestricted* due date in common due date scheduling.

The same diagram to compare the runtimes for different values of the range factor ρ does not lead to any valuable conclusion: the curves are very close and interlaced.

3.3 Special cases

We first consider the case where the penalties are job independent ($\alpha_j = \alpha$ and $\beta_j = \beta$ for all j) (Hoogeveen and van de Velde 1996; Chen and Lin 2004). Indeed, this problem can be efficiently formulated as a MIP with *positional and assignment variables* (Lasserre and Queyranne 1992; Queyranne and Schulz 1994). This formulation is known to have a quite good linear relaxation with a small (polynomial) number of variables. To the best of our knowledge, this formulation has not been applied to irregular criteria. Therefore, it is interesting to test the MIP for this problem. The *assignment* variable y_{jk} is equal to one if job J_j is processed in position k , and zero, otherwise. The *positional* completion time $C_{[k]}$ is the completion time of the job in position k . The positional earliness and tardiness penalties $E_{[k]}$ and $T_{[k]}$ are similarly defined.

$$\min \sum_k \alpha E_{[k]} + \beta T_{[k]}, \quad (6)$$

$$\text{s.t. } T_{[k]} - E_{[k]} = C_{[k]} - \sum_j y_{jk} d_j \quad \forall k; \quad (7)$$

$$\sum_j y_{jk} = 1 \quad \forall k; \quad (8)$$

$$\sum_k y_{jk} = 1 \quad \forall j; \quad (9)$$

$$C_{[k]} \geq \sum_j y_{jk} (r_j + p_j) \quad \forall k; \quad (10)$$

$$C_{[k]} \geq C_{[k-1]} + \sum_j y_{jk} p_j \quad \forall k > 1; \quad (11)$$

$$y_{jk} \in \{0, 1\} \quad \forall j, k; \quad (12)$$

$$E_{[k]} \geq 0, \quad T_{[k]} \geq 0 \quad \forall k. \quad (13)$$

Equations (7) and (13) define the relationship between the positional completion times and the positional earliness and tardiness—observe that $\sum_j y_{jk} d_j$ is the due date of the job in position k . Equations (8) and (9) are the classical assignment constraints. Equations (10) force each job to start after its release dates and (11) require that a job cannot start before its predecessor is completed in the machine sequence.

Table 2 Number of solved instances within 100s when $\alpha_j = \alpha$ and $\beta_j = \beta$

	$\rho = 0.2$	$\rho = 0.4$	$\rho = 0.6$	$\rho = 0.8$	Total
BnB	10/20	13/20	14/20	14/20	51/80
MIP	19/20	5/20	0/20	0/20	24/80

We generated 40-job instances with α and β randomly chosen in $\{1, \dots, 5\}$. For each pair $(\tau, \rho) \in \{0.2, 0.4, 0.6, 0.8\}^2$, five instances were generated which leads to a total of 80 instances. The MIP model was solved by ILOG CPLEX 9.1. Table 2 indicates, for each value of ρ , the number of instances solved within 100s by our algorithm (BnB) and by ILOG CPLEX (MIP). Clearly, the MIP is very efficient to solve instances where all the due dates are in a short time interval because all but one instances with $\rho = 0.2$ can be solved with the MIP formulation while BnB only solves half the instances. Moreover, MIP is usually faster. However, for instances with higher range values, the MIP is no more efficient and the BnB should be preferred.

The algorithm was also tested on a class of difficult instances for the problem $1|r_j|\sum w_j T_j$ (Chu 1992; Akturk and Ozdemir 2000). The best dedicated algorithms for this problem can solve all the instances with 30 up to jobs (Jouglet et al. 2004). Our approach is, of course, less efficient: some instances with 20 and 25 jobs cannot be solved. The main reason of this moderate performance could be that there are several dominances between early jobs that are not taken into account by our algorithm. However, even if earliness penalties are added, the presence of release dates makes these instances more difficult than those generated in Sect. 3.1.

4 Conclusion

This paper presents a new branch-and-bound algorithm for the one-machine earliness-tardiness problem. This algorithm outperforms previous ones because the lower bound is better so that less nodes are required in the search tree. Moreover, new dominance rules also help in reducing the size of the search tree.

Experimental results in Sect. 3.3 suggest that the algorithm could be improved to solve special cases where the range between due dates is narrow and where earliness penalties are significantly smaller than tardiness penalties. The first case could be improved by a dedicated branching scheme while the second case could take advantage of new dominance rules.

From a theoretical point of view, it would be interesting to have a better understanding of the optimal values of the Lagrangean multipliers, in order to have, for example, a performance guarantee on the lower bound. Finally, we would

like to cite that Yau et al. (2006) reported that they solved instances with 40 and 50 jobs when this manuscript was in submission.

Acknowledgements The authors are indebted to an anonymous referee for helpful suggestions and comments. Thanks are also due to Federico della Croce who suggested the authors to test the MIP formulation in Sect. 3.3.

References

- Akturk, M. S., & Ozdemir, D. (2000). An exact approach to minimizing total weighted tardiness with release dates. *IIE Transactions*, 32, 10911101.
- Avella, P., Boccia, M., & DAuria, B. (2005). Near-optimal solutions of large-scale single-machine scheduling problems. *INFORMS Journal on Computing*, 17, 183–191.
- Baker, K. R., & Scudder, G. (1990). Sequencing with earliness and tardiness penalties: a review. *Operations Research*, 38, 22–36.
- Belouadah, H., & Potts, C. N. (1994). Scheduling identical parallel machines to minimize total weighted completion time. *Discrete Applied Mathematics*, 48, 201–218.
- Bülbül, K., Kaminsky, P., & Yano, C. (2007). Preemption in single machine earliness/tardiness scheduling. *Journal of Scheduling* 10, 271–292.
- Chang, P.-C. (1999). A branch and bound approach for single machine scheduling with earliness and tardiness penalties. *Computers & Mathematics with Applications*, 37, 133–144.
- Chen, J. Y., & Lin, S. F. (2004). Minimizing weighted earliness and tardiness penalties in single-machine scheduling with idle time permitted. *Naval Research Logistics*, 49, 760–780.
- Chen, H., Chu, C., & Proth, J. M. (1998). An improvement of the Lagrangean relaxation approach for job shop scheduling: a dynamic programming method. *IEEE Transactions on Robotics and Automation*, 14, 786–795.
- Christofides, N., Alvarez-Valdes, R., & Tamarit, J. M. (1987). Project scheduling with resource constraints: a branch and bound approach. *European Journal of Operational Research*, 29, 262–273.
- Chu, C. (1992). A branch and bound algorithm to minimize total tardiness with different release dates. *Naval Research Logistics*, 39, 265–283.
- De Sousa, J. P., & Wolsey, L. A. (1992). A time-indexed formulation of non-preemptive single-machine scheduling problems. *Mathematical Programming*, 54, 353–367.
- Esteve, B., Aubijoux, C., Chartier, A., & Tkindt, V. (2006). A recovering beam search algorithm for the single machine just-in-time scheduling problem. *European Journal of Operational Research*, 172, 798–813.
- Fisher, M. L. (1976). A dual algorithm for the one-machine scheduling problem. *Mathematical Programming*, 11, 229–251.
- Fry, T. D., Leong, G., & Rakes, T. (1987). Single machine scheduling: a comparison of two solution procedures. *Omega*, 15, 277–282.
- Fry, T. D., Armstrong, R. D., Darby-Dowman, K., & Philipoom, P. R. (1996). A branch and bound procedure to minimize mean absolute lateness on a single processor. *Computers & Operations Research*, 23, 171–182.
- Garey, M. R., Tarjan, R. E., & Wilfong, G. T. (1988). One-processor scheduling with symmetric earliness and tardiness penalties. *Mathematics of Operations Research*, 13, 330–348.
- Gordon, V., Proth, J. M., & Chu, C. (2002). A survey of the state-of-the-art of common due date assignment and scheduling research. *European Journal of Operational Research*, 139, 1–25.
- Hassin, R., & Shani, M. (2005). Machine scheduling with earliness, tardiness and non-execution penalties. *Computers & Operations Research*, 32, 683–705.
- Hendel, Y., & Sourd, F. (2006). Efficient neighborhood search for just-in-time scheduling problems. *European Journal of Operational Research*, 173, 108–119.
- Hoogeveen, J. A. (2005). Multicriteria scheduling. *European Journal of Operational Research*, 167, 592–623.
- Hoogeveen, J. A., & van de Velde, S. L. (1996). A branch-and-bound algorithm for single-machine earliness-tardiness scheduling with idle time. *INFORMS Journal on Computing*, 8, 402–412.
- Jouglet, A., Baptiste, Ph., & Carlier, J. (2004). Branch-and-bound algorithms for total weighted tardiness. In J. Y. T. Leung (Ed.), *Computer & information science series: vol. 13. Handbook of scheduling: algorithms, models and performance analysis*. London: Chapman & Hall/CRC.
- Kaskavelis, C. A., & Caramanis, M. C. (1998). Efficient Lagrangian relaxation algorithms for industry size job-shop scheduling problems. *IIE Transactions*, 30, 1085–1097.
- Kedad-Sidhoum, S., Rios-Solis, Y., & Sourd, F. (2007, in press). Lower bounds for the earliness-tardiness scheduling problem on single and parallel machines. *European Journal of Operational Research*. doi:10.1016/j.ejor.2006.05.052.
- Kim, Y. D., & Yano, C. (1994). Minimizing mean tardiness and earliness in single-machine scheduling problems with unequal due dates. *Naval Research Logistics*, 41, 913–933.
- Lasserre, J.-B., & Queyranne, M. (1992). Generic scheduling polyhedra and a new mixed-integer formulation for single-machine scheduling. In *Proceedings of the 2nd IPCO conference*.
- Lauff, V., & Werner, F. (2004). Scheduling with common due date, earliness and tardiness penalties for multimachine problems: a survey. *Mathematical and Computer Modelling*, 40, 637–655.
- Lenstra, J. K., Rinnooy Kan, A. H. G., & Brucker, P. (1977). Complexity of machine scheduling problems. *Annals of Discrete Mathematics*, 1, 343–362.
- Liaw, C. F. (1999). A branch-and-bound algorithm for the single machine earliness and tardiness scheduling problem. *Computers & Operations Research*, 26, 679–693.
- Luh, P. B., Hoitomt, D. J., Max, E., & Pattipati, K. R. (1990). Schedule generation and reconfiguration for parallel machines. *IEEE Transactions on Robotics and Automation*, 6, 687–696.
- Möhring, R. H., Schulz, A. S., Stork, F., & Uetz, M. (2003). Solving project scheduling problems by minimum cut computations. *Management Science*, 49, 330–350.
- Péridy, L., Pinson, E., & Rivreau, D. (2003). Using short-term memory to minimize the weighted number of late jobs on a single machine. *European Journal of Operational Research*, 148, 591–603.
- Potts, C. N. (1985). A Lagrangean based branch and bound algorithm for single machine sequencing with precedence constraints to minimize total weighted completion time. *Management Science*, 31, 1300–1311.
- Potts, C. N., & Van Wassenhove, L. N. (1983). An algorithm for single machine sequencing with deadlines to minimize total weighted completion time. *European Journal of Operational Research*, 12, 379–387.
- Potts, C. N., & Van Wassenhove, L. N. (1985). A branch and bound algorithm for the total weighted tardiness problem. *Operations Research*, 33, 363–377.
- Queyranne, M., & Schulz, A. S. (1994). *Polyhedral approaches to machine scheduling* (Technical Report 408-1994). Technische Universität Berlin.
- Smith, W. E. (1956). Various optimizers for single-stage production. *Naval Research Logistics*, 3, 59–66.
- Sourd, F. (2004). The continuous assignment problem and its application to preemptive and non-preemptive scheduling with irregular cost functions. *INFORMS Journal on Computing*, 16, 198–208.
- Sourd, F. (2005). Optimal timing of a sequence of tasks with general completion costs. *European Journal of Operational Research*, 165, 82–96.

- Sourd, F. (2006). Dynasearch neighborhood for the earliness-tardiness scheduling problem with release dates and setup constraints. *Operations Research Letters*, 34, 591–598.
- Sourd, F., & Kedad-Sidhoum, S. (2003). The one machine problem with earliness and tardiness penalties. *Journal of Scheduling*, 6, 533–549.
- Tanaka, S., Sasaki, T., & Araki, M. (2003). A branch-and-bound algorithm for the single-machine weighted earliness-tardiness scheduling problem with job independent weights. In *IEEE international conference on systems, man and cybernetics* (pp. 1571–1577).
- Valente, J. M. S., & Alves, R. A. F. S. (2005). An exact approach to early/tardy scheduling with release dates. *Computers & Operations Research*, 32, 2905–2917.
- Verma, S., & Dessouky, M. (1998). Single-machine scheduling of unit-time jobs with earliness and tardiness penalties. *Mathematics of Operations Research*, 23, 930–943.
- Wan, G., & Yen, B. P. C. (2002). Tabu search for single machine scheduling with distinct due windows and weighted earliness/tardiness penalties. *European Journal of Operational Research*, 142, 271–281.
- Yau, H., Pan, Y., & Shi, L. (2006, to appear). New solution approaches to the general single machine earliness-tardiness problem. *IEEE Transactions on Automation Science and Engineering*.